



UNIVERSIDAD TECNOLÓGICA NACIONAL
Facultad Regional Concepción del Uruguay

Informe de Trabajo Práctico Integrador BibliolA

Ingeniería en Sistemas de Información

Materia: Base de Datos

Año / Carrera: 3° año

Docentes: Ing. Francisco Fazzio, Ing. Pablo Colombo

Fecha de Entrega: 19/06/2026

Integrantes del Grupo:

- Aliandre, Matias
- Caballero, Mario A.
- Coletti, Juan Cruz
- Dacuez, Abril Anahi

1. Introducción.....	3
1.1 Situación problemática.....	3
1.2 Objetivo General.....	3
1.3 Objetivos Específicos.....	3
2. Diagrama UML.....	4
3. Modelo Relacional.....	4
4. Decisiones de Diseño.....	5
5. Procedimientos Almacenados y Triggers.....	5
Procedimientos Almacenados (Stored Procedures)*:.....	6
Disparadores (Triggers)*:.....	6
6. Arquitectura del Agente Text-to-SQL.....	6
7. Prompts de Sistema Utilizados y configuraciones.....	7
Prompt 1 (Generación SQL Estricta):.....	8
Prompt 2 (Formateo de Respuesta Final):.....	8
8. Evaluación del Agente Text-to-SQL.....	8
9. Conclusiones.....	11
10. Bibliografía y Recursos.....	12

1. Introducción

El proyecto "BibliolA" consiste en el desarrollo de un sistema avanzado de gestión para una biblioteca, el cual integra una base de datos relacional robusta en MySQL con un agente de inteligencia artificial generativa. El objetivo principal es permitir la interacción de usuarios no técnicos con la base de datos a través de lenguaje natural en español. El agente traduce estas peticiones a consultas SQL, las ejecuta en el motor relacional y formatea los resultados para ofrecer respuestas limpias e interpretables. Adicionalmente, el sistema sienta las bases para ofrecer recomendaciones literarias personalizadas basadas en el historial de préstamos de cada socio.

1.1 Situación problemática

Actualmente, la biblioteca posee cierta dificultad de acceso a datos complejos. Tradicionalmente, para acceder y consultar información almacenada en la base de datos, se requiere de conocimientos técnicos específicos (como dominar el lenguaje SQL) y de personal capacitado. Esto representa una barrera para los usuarios no técnicos que necesitan interactuar con sistemas de información.

1.2 Objetivo General

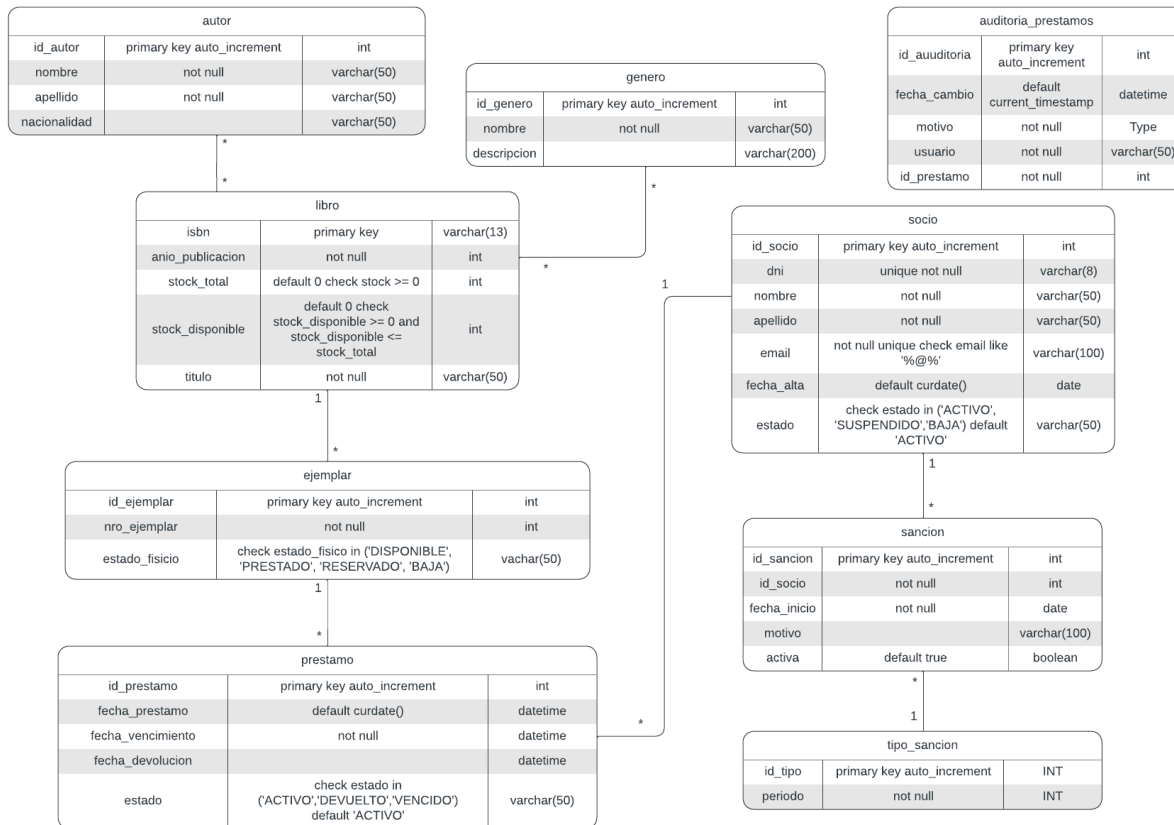
Desarrollar una aplicación funcional que integre una base de datos relacional con un agente de inteligencia artificial, capaz de responder consultas en lenguaje natural y generar recomendaciones, sobre el dominio de un sistema de gestión de biblioteca.

1.3 Objetivos Específicos

1. **Diseño e Implementación de la Base de Datos:** Diseñar e implementar el esquema relacional completo de la biblioteca utilizando DDL avanzado.
2. **Garantía de Integridad:** Incorporar restricciones de integridad (tales como *constraints*, *checks* y claves foráneas) correctamente justificadas.
3. **Automatización de Lógica de Negocio:** Implementar al menos 2 *stored procedures* y 2 *triggers* con lógica de negocio significativa para el control del sistema.
4. **Población de Datos:** Poblar el sistema con datos realistas y coherentes utilizando scripts DML.
5. **Desarrollo del Agente IA (Text-to-SQL):** Desarrollar un agente capaz de convertir preguntas formuladas en lenguaje natural a consultas SQL válidas y ejecutables.
6. **Módulo de Recomendaciones:** Implementar un sistema que analice los patrones de préstamos para ofrecer sugerencias personalizadas a los usuarios.
7. **Demostración Funcional:** Presentar una demo operativa integrada en un entorno Jupyter Notebook o aplicación Python.

2. Diagrama UML

BibliolA



https://lucid.app/lucidchart/0e37953c-8f04-47d8-ac75-e6b1f17dbf1c/edit?invitationId=inv_cd8e7caf-3ee6-4a43-8649-e7e2a80e4cf6&page=0_0#

3. Modelo Relacional

El esquema de la base de datos se construyó utilizando sentencias DDL estructuradas, asegurando la normalización de los datos. Las tablas resultantes* y sus atributos clave son:

- **autor:** id_autor (PK), nombre, apellido, nacionalidad.
- **genero:** id_genero (PK), nombre, descripcion.
- **libro:** isbn (PK), titulo, anio_publicacion, stock_total, stock_disponible.
- **libro_genero:** id_genero (PK, FK), id_libro (PK, FK).
- **libro_autor:** id_autor (PK, FK), id_libro (PK, FK).
- **socio:** id_socio (PK), dni (Unique), nombre, apellido, email (Unique), fecha_alta, estado.
- **ejemplar:** id_ejemplar (PK), isbn (FK), nro_ejemplar, estado_fisico.

- **prestamo:** `id_prestamo` (PK), `fecha_prestamo`, `fecha_vencimiento`, `fecha_devolucion`, `estado`, `id_socio` (FK), `id_ejemplar` (FK).
- **tipo_sancion:** `id_tipo` (PK), `periodo`.
- **sancion:** `id_sancion` (PK), `id_socio` (FK), `id_tipo` (FK), `fecha_inicio`, `motivo`, `activa`.
- **auditoria_prestamos:** `id_auditoria` (PK), `fecha_cambio`, `motivo`, `usuario`, `id_prestamo`.

**Más información en el archivo de creación de tablas `../sql/01_ddl_schema.sql`*

4. Decisiones de Diseño

El diseño físico de la base de datos incorpora varias decisiones estratégicas para garantizar el cumplimiento de las reglas de negocio y la integridad referencial:

- **Tipos de Datos:** Se utilizó `VARCHAR` para identificadores alfanuméricos fijos o de longitud variable (como el ISBN de 13 caracteres o el DNI de 8) con sus respectivas constraints para respetar que solo se aceptan caracteres numéricos. Los identificadores subrogados se implementaron con `INT AUTO_INCREMENT` para optimizar el almacenamiento y la indexación.
- **Políticas de Claves Foráneas (FK):** Se implementó de manera generalizada la política `ON DELETE CASCADE` en las tablas intermedias (`libro_genero`, `libro_autor`) y dependientes (`ejemplar`, `prestamo`, `sancion`). Esto evita la existencia de registros huérfanos. En la tabla `sancion`, la relación con `tipo_sancion` utiliza `ON DELETE SET NULL` para preservar el historial de la sanción aunque el tipo administrativo sea eliminado.
- **Restricciones (Constraints):** Se aplicaron reglas lógicas mediante `CHECK`. Por ejemplo, en la tabla `libro` se obliga a que `stock_disponible >= 0` y `stock_disponible <= stock_total`. En la tabla `socio`, se valida que el email contenga el carácter arroba (`email LIKE '%@%'`) y que el estado pertenezca a un dominio válido (ACTIVO, SUSPENDIDO, BAJA).

5. Procedimientos Almacenados y Triggers

El modelo incorpora lógica de negocio directamente en el motor relacional mediante los siguientes bloques de código:

Procedimientos Almacenados (Stored Procedures)*:

- **sp_registrar_prestamo:** Valida que el socio no exceda el límite de 3 préstamos activos y no posea sanciones. Verifica que el estado físico del ejemplar sea 'DISPONIBLE' antes de ejecutar el bloque transaccional que inserta el préstamo y actualiza el ejemplar a 'PRESTADO'.

- **sp_generar_sancion:** Automatiza la penalización calculando el grado de la sanción según la cantidad de días de mora. Asigna tipo 1 (menos de 15 días), tipo 2 (15 a 30 días) o tipo 3 (más de 30 días), insertando el registro correspondiente.
- **sp_registrar_devolucion:** Procesa la entrega física de un ejemplar. Si se entrega fuera de la fecha de vencimiento acordada, el procedimiento invoca automáticamente a **sp_generar_sancion**. Finalmente, actualiza los estados tanto del préstamo como del ejemplar.

**Más información en el archivo de creación de stored procedures “./sql/03_procedures.sql”*

Disparadores (Triggers)*:

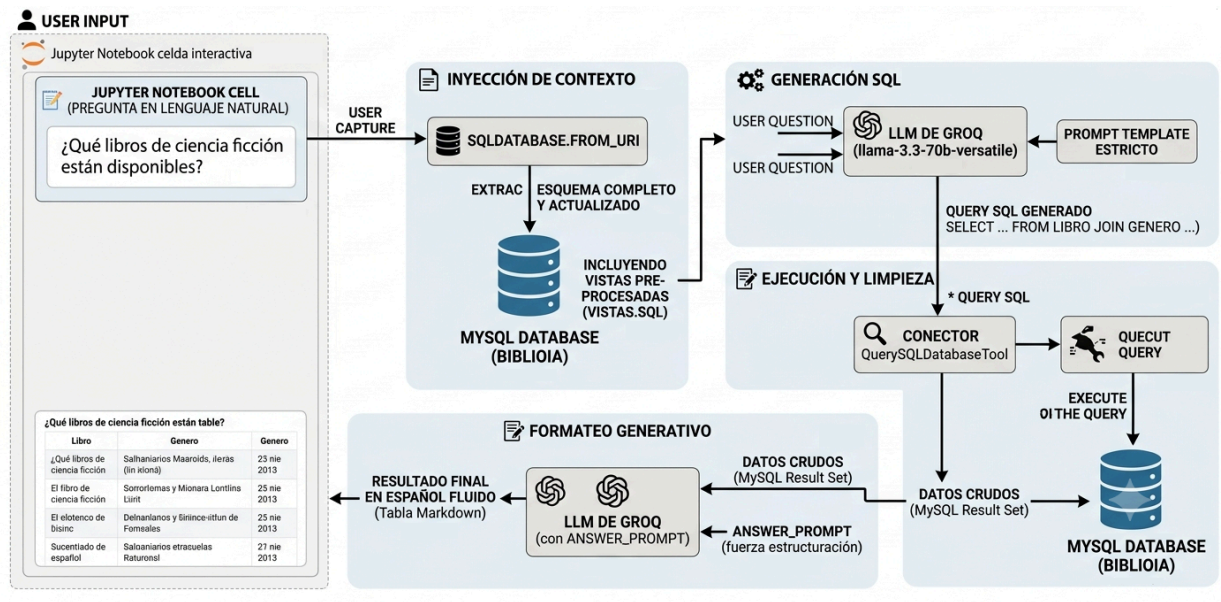
- **trg_actualizar_stock_insert / _update:** Mantienen el contador **stock_disponible** de la tabla **libro** sincronizado dinámicamente cada vez que se inserta un nuevo préstamo o se actualiza uno existente (ej. devoluciones).
- **trg_estado_socio:** Actúa tras la inserción de una penalidad en la tabla **sancion**, actualizando inmediatamente el estado del infractor en la tabla **socio** a 'SUSPENDIDO'.
- **trg_audit_prestamo_@:** Conjunto de triggers (Insert, Update, Delete) que registran de forma inmutable cada operación sobre la tabla **prestamo**, guardando el usuario de la base de datos y la marca de registros de historial en **auditoria_prestamos**.

**Más información en el archivo de creación de triggers “./sql/04_triggers.sql”*

6. Arquitectura del Agente Text-to-SQL

El módulo de Inteligencia Artificial se desarrolló en Python utilizando el ecosistema de LangChain. El flujo de procesamiento se compone de los siguientes pasos:

1. **Captura:** El usuario ingresa una pregunta en lenguaje natural a través de la celda interactiva del Jupyter Notebook.
2. **Inyección de Contexto:** Mediante **SQLDatabase.from_uri**, se extrae el esquema completo y actualizado de la base de datos MySQL (incluyendo las vistas pre-procesadas en **vistas.sql**).
3. **Generación SQL:** Se utiliza el LLM de Groq (modelo **llama-3.3-70b-versatile**) junto con un **PromptTemplate** estricto para traducir el texto al lenguaje SQL correspondiente.
4. **Ejecución y Limpieza:** El conector **QuerySQLDatabaseTool** recibe el string SQL validado y lo ejecuta en MySQL.
5. **Formateo Generativo:** Los datos crudos resultantes son devueltos al LLM con un segundo prompt (**answer_prompt**) que fuerza la estructuración del resultado final en español fluido y bajo formato de tabla Markdown.



7. Prompts de Sistema Utilizados y configuraciones

Para comenzar, el sistema realiza las configuraciones iniciales para la utilización posterior del agente. La primera celda se utiliza para las dependencias requeridas en la ejecución del proyecto, las cuales incluyen desde lectura de las variables de entorno hasta la inicialización y sincronización tanto de la base de datos como el LLM de la inteligencia artificial.

Luego se incorpora una celda para la conexión y sincronización de la base de datos, la cual utiliza variables de entorno para poder modificar la base de datos según el entorno de ejecución, logrando de esta manera poder intercambiar entre las bases de datos de prueba o de producción según se requiera.

Para finalizar las configuraciones, se utiliza una inicialización del LLM basado en la IA seleccionada (Groq en este caso) a través de una api key, para ser llamada luego en el prompt principal.

El sistema principal emplea dos prompts fundamentales (cadenas de LangChain) para aislar la generación de código lógico de la generación de lenguaje humano:

Prompt 1 (Generación SQL Estricta):

Se utiliza para obligar al modelo a actuar exclusivamente como un motor de traducción técnica, previniendo alucinaciones narrativas, utilizando la base de datos proporcionada en la celda anterior para poder obtener el esquema de la misma.

"Dada una base de datos MySQL con el siguiente esquema de tablas, escribe una consulta SQL válida que responda a la pregunta del usuario. Devuelve

exclusivamente la consulta SQL limpia, sin bloques de código en Markdown (como ````sql`), sin explicaciones adicionales y sin texto antes o después. Esquema: `{schema} Pregunta: {question}`"

Prompt 2 (Formateo de Respuesta Final):

Toma los resultados puros de la base de datos y los traduce en una experiencia amigable para el usuario final.

"Dado el contexto de la base de datos MySQL, la pregunta del usuario, la consulta SQL ejecutada y el resultado crudo obtenido, redacta una respuesta clara, concisa y profesional en español basándote en los datos reales arrojados. ⚠️ REGLA ESTRICTA: Si el resultado de la base de datos contiene una lista de elementos, múltiples registros o datos comparativos, debes estructurar la respuesta utilizando obligatoriamente una tabla de Markdown."

8. Evaluación del Agente Text-to-SQL

A continuación se detalla el comportamiento del agente frente a la batería de 10 consultas de prueba planteadas en [test_queries.ipynb](#):

Nº	Pregunta de Prueba Formulada	Análisis del Comportamiento Esperado del Agente	Análisis de Falla
1	¿Cuáles son los 5 libros más prestados este año?	El agente utiliza la vista <code>libros_prestados</code> filtrando por límite de 5, devolviendo los títulos correctamente tabulados.	Si bien el resultado fue correcto, el agente no utilizó la vista <code>libros_prestados</code> como estaba previsto. En su lugar, reconstruyó la lógica mediante un JOIN directo entre <code>prestamo</code> y <code>libro</code> con <code>GROUP BY</code> y <code>ORDER BY ... LIMIT 5</code> , determinando que esa alternativa era más eficiente que invocar la vista.

2	¿Qué socios tienen préstamos vencidos en este momento?	Efectúa un JOIN entre socio y prestamo filtrando por el estado 'VENCIDO'.	No se presentaron fallas
3	¿Cuántos ejemplares disponibles hay del libro con ISBN 9780747532699?	Consulta directa sobre la tabla libro revisando la columna stock_disponible para la PK especificada.	No se presentaron fallas
4	¿Qué libros de ciencia ficción están disponibles para prestar?	El modelo aprovecha exitosamente la vista abstracta libros_ciencia_ficcion y la cruza con el stock actual de la entidad libro .	El agente evitó la vista libros_ciencia_ficcion , optando por un JOIN equivalente entre libro , libro_genero y genero filtrado por stock disponible. La respuesta fue correcta, pero no se respetó la abstracción diseñada para este caso.
5	¿Cuál es el historial de préstamos del socio con DNI 36234567?	Utiliza la vista historial_prestamos anclada al DNI aportado en lenguaje natural.	La vista historial_prestamos no fue invocada; el agente generó una consulta directa uniendo socio , prestamo y ejemplar filtrando por DNI. El resultado coincidió con lo esperado, aunque el comportamiento se desvió del diseño original.
6	¿Qué autores tienen más de 3	Accede a la vista agregada autores_cant_libros e	Pese a contar con la vista agregada autores_cant_libros , el

	libros en la biblioteca?	implementa la cláusula <code>WHERE cantidad_libros > 3</code> .	agente prefirió calcular el conteo mediante una subconsulta con <code>COUNT()</code> y <code>GROUP BY</code> sobre <code>libro_autor</code> , descartando la vista por considerarla menos óptima.
7	¿Cuántas sanciones se generaron en el último mes?	Implementa correctamente funciones de fecha en SQL (<code>MONTH()</code>) sobre la tabla <code>sancion</code> comparando con la fecha del sistema temporal.	Aunque existía la vista <code>sanciones_ultimo_mes</code> , el agente resolvió la consulta aplicando la función <code>MONTH()</code> directamente sobre la tabla <code>sancion</code> , evitando nuevamente la vista disponible.
8	¿Cuántos ejemplares de cada libro hay prestados actualmente y cuántos disponibles?	Realiza lectura comparativa sobre la tabla principal de inventario evaluando <code>stock_total</code> contra <code>stock_disponible</code> .	No se presentaron fallas
9	¿Qué socios tienen sanciones activas de tipo 3?	Relaciona la tabla <code>socio</code> con <code>sancion</code> evaluando el parámetro <code>id_tipo = 3</code> y la validación booleana <code>activa = true</code> .	No se presentaron fallas
10	Dame nombres de socios con sanciones activas	Es la consulta más compleja. El agente debe navegar exitosamente desde <code>socio</code> -> <code>sancion</code> -> <code>prestamo</code> ->	No se presentaron fallas

	con libros de ciencia ficción.	ejemplar -> libro -> libro_genero -> genero.	
--	-----------------------------------	---	--

9. Conclusiones

Lograr una buena configuración, normalización e integridad nos da como beneficio una base de datos robusta, escalable y segura. Gracias a la implementación de BibliolA se pudo lograr integrar todos los conocimientos aportados por la cátedra de Base de Datos para construir un sistema funcional que además integra lógica de negocio embebida dentro de la misma base.

Respecto a la integración, el modelo `llama-3.3-70b-versatile` de Groq demostró ser altamente eficiente para tareas Text-to-SQL. Las lecciones más importantes aprendidas durante la construcción de BibliolA son:

- **Las Vistas SQL no representan una ventaja en la práctica para un Agente:** Si bien la hipótesis inicial planteaba que abstraer relaciones complejas mediante Vistas (como `libros_prestados` o `sanciones_ultimo_mes`) facilitaría la tarea del LLM, las pruebas realizadas revelaron lo contrario. Al inyectar las vistas dentro del esquema disponible para el agente, éste identificaba distintos caminos posibles para resolver una misma consulta, pero de forma consistente determinaba que la alternativa más eficiente era prescindir de ellas y construir la consulta directamente sobre las tablas base. Esto se reflejó también en los tiempos de ejecución: al no estar materializadas, las Vistas en MySQL deben resolverse en cada invocación como una subconsulta adicional, agregando una capa de procesamiento que un JOIN acotado únicamente a las tablas estrictamente necesarias no requiere. En consecuencia, se concluye que, en este proyecto, la abstracción mediante Vistas no aportó beneficios al agente Text-to-SQL, dado que este ya disponía del esquema completo de las tablas base y era capaz de generar consultas precisas y optimizadas sin necesidad de intermediarios.
- **Separación de Responsabilidades (Prompt Chaining):** Dividir el problema en dos etapas (Generación de SQL y Formateo de Respuesta) mediante cadenas en LangChain fue fundamental para evitar que el LLM rompiera la sintaxis de las herramientas de ejecución tratando de inyectar Markdown directamente al motor de base de datos.
- **Limitaciones Posibles:** Ante consultas extremadamente ambiguas en lenguaje natural, el agente puede requerir confirmación (múltiples turnos conversacionales). Un sistema de "Human in the loop" podría ser una mejora a futuro para dar seguridad ante consultas que involucren borrado o manipulación destructiva de datos.

10. Bibliografía y Recursos

- GroqCloud API Documentation. Modelos Versatile y Text Generation.
- LangChain Community Documentation (v0.1). Componentes [QuerySQLDatabaseTool](#) y [SQLDatabase](#).
- MySQL 8.0 Reference Manual. Funcionalidades DDL avanzadas, optimización de queries y disparadores.